
STATIC OVERLOAD RESOLUTION AND MONOMORPHIZATION FOR FUNCTIONAL LANGUAGES

THOMAS VERGNE 

Unaffiliated

e-mail address: contact@thomas-vergne.fr

ABSTRACT. We present a formal framework for the static resolution of function overloading in an extension of the simply typed lambda-calculus, coupled with a construction relation via global monomorphization. Our system reconciles the flexibility of local extension declarations with a deterministic selection of the most specific implementation, structurally eliminating the resolution indirection associated with passing dictionaries or dispatch tables. We formalize the syntax, the declarative type system, and the specialization semantics of this calculus, and we establish mechanized metatheoretical guarantees in Coq/Rocq: type uniqueness, type preservation under elaboration, forward semantic simulation, and the End-to-End Soundness theorem.

1. INTRODUCTION

Dealing with function overloading is now a well-known problem, but also a major technical challenge in the design of type systems. Formally distinguished from parametric polymorphism and subtype inclusion by Cardelli and Wegner [CW85], ad-hoc polymorphism allows the same identifier to be associated with multiple semantics depending on the type of the arguments. Generally, efficient management of function overloading occurs over multiple compilation passes to minimize the overhead caused by this overloading. In languages such as Haskell, function overloading is managed using a system of *type classes* [WB89], and resolved at compile time by a mechanism that *dictionary-passing* [Aug93]. Subsequent specialization and inlining passes can eliminate some or all of this indirection in cases where these optimizations are applicable.

The issue we are addressing here is the handling of overloading in statically typed functional programming languages, and more specifically, what happens to these overloaded functions during machine code generation. The ultimate goal is to generate direct code free of any references to dictionaries or virtual execution tables, while preserving the safety guarantees of our type system to the greatest extent possible. Unlike dynamic approaches to overloaded operators with late binding [Sav10], we aim for a completely static resolution using global monomorphization.

The current limitations of the scientific literature often boil down to a trade-off: on the one hand, class-based systems require overloading to occur within implicit structures resolved by dictionaries, involving complex specialization passes to eliminate indirection at

Key words and phrases: Function Overloading, Monomorphization, Type Systems, Operational Semantics, Mechanized Proofs, Coq.

runtime; on the other hand, ad-hoc overloading in languages such as C++ [KS13] offers welcome syntactic flexibility, but its resolution mechanism is based on a complex set of informal rules, not designed around the formally verified meta-theoretical framework that we present here.

Thus, the central issue addressed in this article is how to provide expressive and ergonomic function overloading while ensuring a deterministic choice of implementation and the complete elimination of overloading through formally verified monomorphization. Let us clarify a terminological convention from the outset: in this article, we use the term *monomorphization* to refer to the static specialization of ad-hoc polymorphism instances; property-independent parametric polymorphism (**let**) is, however, preserved in the target calculus.

1.1. Contributions. Our contributions are as follows:

- **Surface Calculus $\lambda_{\mathcal{O}}$** : We define an extended functional calculus equipped with explicit property constructions and local extensions for overloading.
- **Deterministic Resolution under WFC** : We formalize a selection criterion for the most specific extension (MSC) and prove that overloading resolution is deterministic under an explicit condition for the correct formation of the catalog (WFC), while establishing the uniqueness of the source typing.
- **Monomorphization Elaboration** : We define an inductive construction relation $\Omega \vdash e \hookrightarrow e_m$ that translates any source program into a target calculus $\lambda_{\mathcal{M}}$ free of ad hoc polymorphism.
- **Formality Guarantees and End-to-End Soundness** : We demonstrate type preservation, forward semantic simulation, and an end-to-end soundness theorem, which guarantees that no target program reaches a deadlock state.
- **Integrated Mechanization** : The entire metatheory (syntax, typing, specialization, and safety theorems) is fully automated and verified without axioms in the Coq/Rocq proof assistant.

2. MOTIVATION

The main motivation for developing this system and this syntax is to allow end users to define overloaded functions in an intuitive and natural way, without having to deal with complex constraint systems, such as those found in Haskell [WB89] or Rust. Recent work, such as that by Beneš and Brachthäuser [BB25], highlights this historical tension between Hindley-Milner-style type checking and ad-hoc overloading resolution, advocating for a clearer separation between constraint collection and implementation choice.

The goal, therefore, is to simplify the program-writing process as a first step, while ensuring the formal correctness of the source typing and the target monomorphized computation; to provide a solid formal foundation for our approach; and to structurally eliminate any overhead from indirection at runtime (without requiring dictionary lookup or runtime type inspection). We emphasize that our contribution focuses on the formal modeling and automated verification of this compilation chain; low-level optimizations and empirical measurements of execution throughput are the domain of specific compilers and are orthogonal to our semantic formalization.

Thus, the system we describe does not merely propose a new approach to solving implicit constraints, but rather offers an approach similar to current methods, albeit one that differs in its simplicity of implementation.

3. BACKGROUND

We take the λ -calculus equipped with a declarative, Hindley–Milner-style polymorphic typing system with explicit instantiation [Rém02] as the formal foundation for our programming language. Numerous extensions of this formal framework have been proposed to incorporate overloading, notably Jones’s system of qualified types [Jon95], as well as the systems combining HM(X) inference and constraint handling rules (CHRs) proposed by Stuckey and Sulzmann [SS05]. Other foundations are based on overloaded abstraction calculi with early or late binding, such as Castagna’s $\lambda\{\}$ [Cas95] or context-dependent overloading approaches [CF98].

We will enhance this model with additional constructs to model a more expressive and realistic language. In particular, we will develop a type system with constructs such as **properties** and **extensions**, which allow us to model function overloading in our programming language.

Definition 3.1 (Properties). A property is defined as an abstract association between a function name and a function type. It is the direct analogue of the *type classes* introduced by Wadler and Blott [WB89]. They are maintained and managed in the same **environment** as **traditional variables**.

Definition 3.2 (Extensions). An extension is a concrete implementation of a property for a given type. In this case, it is the direct analogue of *instances* in Haskell. These types of extensions are intended to be type-checked without altering the behavior of the type system. They will primarily be used during the monomorphization phase.

We will use the term $\lambda_{\mathcal{O}}$ to refer to our programming language, which is an extension of simply typed lambda calculus with property and extension constructs. We thus gradually arrive at an ML-style programming language, with basic constructs, functions, products, and properties and extensions to model function overloading.

4. SYNTAX DEFINITION

Since our language is based on an enriched λ -calculus, the various language constructs are expressions, including local property declarations (**prop**) and overloaded extension calls (**ext**). This is a form of explicit local overloading. It is important to note that properties do not have their own reduction semantics in the operational semantics: they have only a static effect on the typing environment Δ and are removed during evaluation via monomorphization. Extension declarations and lexical variables, on the other hand, are maintained in separate environments due to the different nature of their management. We present the syntax of our language $\lambda_{\mathcal{O}}$ in Figure 1.

Type	τ	$::=$	nat	
			bool	
			string	
			$\tau_1 \rightarrow \tau_2$	function type
			$\tau_1 \times \tau_2$	product type
			α	type variable
Expr	e	$::=$	$x[\vec{\tau}]$	variable
			ext $f[\vec{\tau}]$	extension
			v	literal value
			$\lambda[x:\tau].e$	lambda abstraction
			$e_1 e_2$	function application
			$e_1 + e_2$	addition
			let $x(\vec{\alpha}):\tau = e_1$ in e_2	let binding
			prop $f(\vec{\alpha}):\tau$ in e	property
			(e_1, e_2)	product construction
			$\pi_1 e$	first projection
			$\pi_2 e$	second projection
			if e_1 then e_2 else e_3	conditional
Value	v	$::=$	0	
			true	
			"Hello world"	
			...	

Figure 1: $\lambda_{\mathcal{O}}$'s Language Syntax

In our language $\lambda_{\mathcal{O}}$, we equip the λ -calculus with basic primitive operations such as character strings, integers, Booleans, independent products, conditionals, property declarations (**prop**), and call sites overloaded by extensions (**ext** $f[\vec{\tau}]$). Extension implementations, in turn, are declared and collected in the global catalog C .

In this syntax, we represent let-bindings and property declarations in such a way that they can carry explicit type abstractions. The vector notation $\vec{\alpha} = (\alpha_1, \dots, \alpha_k)$ denotes the ordered sequence of bound type variables. Similarly, $\vec{\tau} = (\tau_1, \dots, \tau_k)$ corresponds to the tuple of concrete types applied during the instantiation of the polymorphic signature. The instantiation of a schema $\sigma = \forall \vec{\alpha}. \tau_0$ on a vector $\vec{\tau}$ is a well-defined functional application denoted by $\text{INST}(\sigma, \vec{\tau}) = \tau$.

We distinguish between two categories of identifiers from a syntactic perspective: standard lexical variables (x, y, z) manipulated via the lexical context Γ , and symbols for properties and overloaded functions (f, g, h) declared in Δ and resolved via the extension catalog C at the call site **ext** $f[\vec{\tau}]$. The global conventions for the meta-variables used throughout the paper are summarized in Table 1.

Meta-variable	Domain	Description / Status
x, y, z	Var	Lexical variables
f, g, h	PropName	Property names and overloaded functions
α, β	TyVar	Type variables
$\vec{\alpha} = (\alpha_1, \dots, \alpha_k)$	TyVar^k	Vectors of bound type variables
τ	Type	Monomorphic types
$\vec{\tau} = (\tau_1, \dots, \tau_k)$	Type^k	Vectors of concrete instantiation types
$\sigma = \forall \vec{\alpha}. \tau$	Scheme	Polymorphic type schemes
Γ	Var $\rightarrow$ Scheme	Source lexical context
Δ	PropName $\rightarrow$ Scheme	Declared property signatures context
$C \subseteq \text{PropName} \times \text{Scheme} \times \text{Expr}_{\mathcal{M}}$	Catalog	Global catalog of candidate implementations
$\Omega \subseteq \text{PropName} \times \text{Type}^k \times \text{Expr}_{\mathcal{M}}$	InstEnv	Concrete specialization table
Γ_m	Var $\rightarrow$ Scheme	Target lexical context in $\lambda_{\mathcal{M}}$

Table 1: Summary of meta-variables and syntax conventions

5. TYPE SYSTEM

Given the nature of our syntax, we present a declarative, Hindley–Milner-style polymorphic typing system with explicit instantiation. The typing rules for our source language $\lambda_{\mathcal{O}}$ are shown in Figure 2, where a type proof is denoted by $\Gamma, \Delta \vdash_{\mathcal{O}} e : \tau$.

The goal is to model an expressive functional calculus using standard structures while maintaining a fully mechanizable and transparent meta-theory.

We emphasize the positioning of our type system: our formalization does not propose a constraint inference, unification, or type synthesis algorithm in the Hindley–Milner style, nor does it establish an existence theorem for inference. Unlike heavy-duty algorithmic frameworks (such as HM(X) [SS05] or semi-unification [FV05]), our system deliberately adopts a declarative formulation in which generic bindings and instantiation types $[\vec{\tau}]$ are explicitly annotated. This design clearly decouples static typing (which guarantees the uniqueness of the derived type, Theorem 5.14) from overloading resolution and global monomorphization. We distinguish between $(\forall)$ -type schemas and simple monomorphic types, in the style of Hindley–Milner systems.

Our type system for $\lambda_{\mathcal{O}}$ is equipped with two environments: Γ for local lexical variables and Δ for the signatures of overloading properties.

$$\begin{array}{c}
\frac{x : \sigma \in \Gamma \quad \text{INST}(\sigma, \vec{\tau}) = \tau}{\Gamma, \Delta \vdash_{\mathcal{O}} x[\vec{\tau}] : \tau} \text{T-VAR} \qquad \frac{}{\Gamma, \Delta \vdash_{\mathcal{O}} n : \text{nat}} \text{T-NUM} \\
\\
\frac{}{\Gamma, \Delta \vdash_{\mathcal{O}} s : \text{string}} \text{T-STRING} \qquad \frac{}{\Gamma, \Delta \vdash_{\mathcal{O}} b : \text{bool}} \text{T-BOOL} \\
\\
\frac{\Gamma \cup \{x : \tau_1\}, \Delta \vdash_{\mathcal{O}} e : \tau_2}{\Gamma, \Delta \vdash_{\mathcal{O}} \lambda[x : \tau_1].e : \tau_1 \rightarrow \tau_2} \text{T-ABS} \qquad \frac{\Gamma, \Delta \vdash_{\mathcal{O}} e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma, \Delta \vdash_{\mathcal{O}} e_2 : \tau_1}{\Gamma, \Delta \vdash_{\mathcal{O}} e_1 \ e_2 : \tau_2} \text{T-APP} \\
\\
\frac{\Gamma, \Delta \vdash_{\mathcal{O}} e_1 : \text{nat} \quad \Gamma, \Delta \vdash_{\mathcal{O}} e_2 : \text{nat}}{\Gamma, \Delta \vdash_{\mathcal{O}} e_1 + e_2 : \text{nat}} \text{T-ADD} \\
\\
\frac{\Gamma, \Delta \vdash_{\mathcal{O}} e_1 : \text{bool} \quad \Gamma, \Delta \vdash_{\mathcal{O}} e_2 : \tau \quad \Gamma, \Delta \vdash_{\mathcal{O}} e_3 : \tau}{\Gamma, \Delta \vdash_{\mathcal{O}} \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{T-IF} \\
\\
\frac{\Gamma, \Delta \vdash_{\mathcal{O}} e_1 : \tau_1 \quad \Gamma, \Delta \vdash_{\mathcal{O}} e_2 : \tau_2}{\Gamma, \Delta \vdash_{\mathcal{O}} (e_1, e_2) : \tau_1 \times \tau_2} \text{T-PAIR} \qquad \frac{\Gamma, \Delta \vdash_{\mathcal{O}} e : \tau_1 \times \tau_2}{\Gamma, \Delta \vdash_{\mathcal{O}} \pi_1 e : \tau_1} \text{T-FIRST} \\
\\
\frac{\Gamma, \Delta \vdash_{\mathcal{O}} e : \tau_1 \times \tau_2}{\Gamma, \Delta \vdash_{\mathcal{O}} \pi_2 e : \tau_2} \text{T-SECOND} \\
\\
\frac{(\forall \vec{\tau}. \text{INST}(\forall \vec{\alpha}. \tau_1, \vec{\tau}) = \tau' \implies \Gamma, \Delta \vdash_{\mathcal{O}} e_1 : \tau') \quad \Gamma \cup \{x : \forall \vec{\alpha}. \tau_1\}, \Delta \vdash_{\mathcal{O}} e_2 : \tau_2}{\Gamma, \Delta \vdash_{\mathcal{O}} \text{let } x[\vec{\alpha}] = e_1 \text{ in } e_2 : \tau_2} \text{T-LET} \\
\\
\frac{f : \sigma \in \Delta \quad \text{INST}(\sigma, \vec{\tau}) = \tau}{\Gamma, \Delta \vdash_{\mathcal{O}} \text{ext } f[\vec{\tau}] : \tau} \text{T-OVER} \qquad \frac{\Gamma, \Delta \cup \{f : \forall \vec{\alpha}. \tau_1\} \vdash_{\mathcal{O}} e_2 : \tau_2}{\Gamma, \Delta \vdash_{\mathcal{O}} \text{prop } f[\vec{\alpha}] : \tau_1 \text{ in } e_2 : \tau_2} \text{T-PROP}
\end{array}$$

Figure 2: Type system rules for the language $\lambda_{\mathcal{O}}$

Before defining the properties of our type system, we must consider two important constructs: operational semantics to represent step-by-step reduction, and the notion of a value to represent expressions that can no longer be reduced. These are so important that the proofs of our soundness properties will be based on these two constructs in addition to the typing rules. We will present the operational semantics of our language $\lambda_{\mathcal{O}}$ in Figure 3, and the notion of a value in Figure 4.

$$\begin{array}{c}
 \frac{\text{value } e_1 \quad \text{value } e_2}{(\lambda[x: \tau].e_1) \ e_2 \longrightarrow e_1[x := e_2]} \text{ST-APPABS} \qquad \frac{e_1 \longrightarrow e'_1}{e_1 \ e_2 \longrightarrow e'_1 \ e_2} \text{ST-APP1} \\
 \\
 \frac{\text{value } e_1 \quad e_2 \longrightarrow e'_2}{e_1 \ e_2 \longrightarrow e_1 \ e'_2} \text{ST-APP2} \qquad \frac{n_1, n_2 \in \mathbb{N}}{n_1 + n_2 \longrightarrow n_3} \text{ST-ADDCONST} \quad \text{where } n_3 = n_1 + n_2 \\
 \\
 \frac{e_1 \longrightarrow e'_1}{e_1 + e_2 \longrightarrow e'_1 + e_2} \text{ST-ADD1} \qquad \frac{\text{value } e_1 \quad e_2 \longrightarrow e'_2}{e_1 + e_2 \longrightarrow e_1 + e'_2} \text{ST-ADD2} \\
 \\
 \frac{e_1 \longrightarrow e'_1}{\text{let } x = e_1 \text{ in } e_2 \longrightarrow \text{let } x = e'_1 \text{ in } e_2} \text{ST-LET} \\
 \\
 \frac{\text{value } v}{\text{let } x = v \text{ in } e_2 \longrightarrow e_2[x := v]} \text{ST-LETVALUE} \\
 \\
 \frac{\text{value } \text{true}}{\text{if } \text{true} \text{ then } e_2 \text{ else } e_3 \longrightarrow e_2} \text{ST-IFTRUE} \\
 \\
 \frac{\text{value } \text{false}}{\text{if } \text{false} \text{ then } e_2 \text{ else } e_3 \longrightarrow e_3} \text{ST-IFFALSE} \\
 \\
 \frac{e_1 \longrightarrow e'_1}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \longrightarrow \text{if } e'_1 \text{ then } e_2 \text{ else } e_3} \text{ST-IF} \qquad \frac{e_1 \longrightarrow e'_1}{(e_1, e_2) \longrightarrow (e'_1, e_2)} \text{ST-PAIR1} \\
 \\
 \frac{\text{value } e_1 \quad e_2 \longrightarrow e'_2}{(e_1, e_2) \longrightarrow (e_1, e'_2)} \text{ST-PAIR2} \qquad \frac{\text{value } v_1 \quad \text{value } v_2}{\pi_1(v_1, v_2) \longrightarrow v_1} \text{ST-FIRSTPAIR} \\
 \\
 \frac{\text{value } v_1 \quad \text{value } v_2}{\pi_2(v_1, v_2) \longrightarrow v_2} \text{ST-SECONDPAIR} \qquad \frac{e \longrightarrow e'}{\pi_2 e \longrightarrow \pi_2 e'} \text{ST-SECOND} \\
 \\
 \frac{e \longrightarrow e'}{\pi_1 e \longrightarrow \pi_1 e'} \text{ST-FIRST}
 \end{array}$$

 Figure 3: Small step operational semantics for the language $\lambda_{\mathcal{O}}$

In these relatively standard definitions of operational semantics, the reduction rules for expressions of the language $\lambda_{\mathcal{O}}$ are defined. We have deliberately omitted the reduction rules for properties, as they are supposed to be invisible from the perspective of operational semantics. In fact, these rules merely extend the scope of overloaded functions.

$$\begin{array}{c}
\frac{n \in \mathbb{N}}{\mathbf{value} \ n} \text{V-CONST} \qquad \frac{b \in \{\mathit{true}, \mathit{false}\}}{\mathbf{value} \ b} \text{V-BOOL} \qquad \frac{s \in \text{String}}{\mathbf{value} \ s} \text{V-STRING} \\
\\
\frac{}{\mathbf{value} \ \lambda[x:\tau].e} \text{V-LAM} \qquad \frac{\mathbf{value} \ e_1 \quad \mathbf{value} \ e_2}{\mathbf{value} \ (e_1, e_2)} \text{V-PAIR}
\end{array}$$

Figure 4: Most primitive values for the language $\lambda_{\mathcal{O}}$

These values are considered values, meaning they cannot be further reduced. Thus, any expression that reduces to one of these values is considered a final value.

5.1. Proofs & Coq Mechanization. To begin the proof of the theorems essential to the soundness of our entire system, we must note that all the theorems presented in this section and in subsequent sections have been formulated and proven in Coq¹. The mechanization of overloaded type checking and its handling in proof assistants is an area of growing interest, as evidenced by recent work by Charguéraud et al. [CBR24].

5.2. Type System Properties. We must now present the properties of our type system for the source language $\lambda_{\mathcal{O}}$. We have defined the most essential theorems to guarantee the soundness of our type system as well as the uniqueness of the chosen types modulo the "most specific type" property. Here are the main properties we need to prove:

5.2.1. Useful Lemmas.

Lemma 5.1 (Type environment weakening). *Variables can always be added in full at the end of the variable context without changing its differentiability. In other words, $\forall \Gamma \Gamma' \Delta e \tau. \Gamma, \Delta \vdash_{\mathcal{O}} e : \tau \implies \Gamma \cup \Gamma', \Delta \vdash_{\mathcal{O}} e : \tau$. Here, $\Gamma \cup \Gamma'$ denotes the concatenation of Γ and Γ' .*

Lemma 5.2 (Overloaded environment weakening). *Similarly, when an expression e is well-typed under Δ and $\Delta \subseteq \Delta'$, then e retains its type under Δ' . In other words, $\forall \Delta \Delta' \Gamma e \tau. \Gamma, \Delta \vdash_{\mathcal{O}} e : \tau \wedge \Delta \subseteq \Delta' \implies \Gamma, \Delta' \vdash_{\mathcal{O}} e : \tau$.*

Proving these two lemmas generally involves generalizing the context Δ' or Γ' in order to establish a recurrence relation on the derivation of $\Gamma, \Delta \vdash_{\mathcal{O}} e : \tau$ and applying the recurrence assumptions.

We assume that for any term e of type τ under empty contexts satisfying the predicate **value** e , the term e corresponds to the value constructor of type τ (for example, if $\emptyset, \emptyset \vdash_{\mathcal{O}} e : \text{bool}$ and **value** e , then $e = \mathit{true}$ or $e = \mathit{false}$).

These lemmas are proven by double recurrence on the structure of **value** e and on the derivation of $\emptyset, \emptyset \vdash_{\mathcal{O}} e : \tau$.

Theorem 5.3 (Expression progression theorem). *Let e be a term of type τ that is free of any properties. If e has type τ under empty contexts, then e is either a value, or there exists e' such that $e \longrightarrow e'$:*

$$\emptyset, \emptyset \vdash_{\mathcal{O}} e : \tau \implies \mathbf{value} \ e \vee \exists e'. e \longrightarrow e'$$

¹<https://github.com/thomasvergne/easy-and-yet-powerful-overloading>

This theorem is one of the fundamental building blocks of operational semantics and type system safety. It can be proven by recurrence on the derivation of $\emptyset, \emptyset \vdash_{\mathcal{O}} e : \tau$, using the canonical form lemmas.

One has traditionally the properties of permutation and hiding by prefix on the contexts, which are established by structural induction on the typing judgments.

Lemma 5.4 (Typing preservation with substitution). *Let $\Gamma, \Delta, x, \sigma, e, v$, and τ be given. If $\Gamma \cup \{x : \sigma\}, \Delta \vdash_{\mathcal{O}} e : \tau$, and if for every vector $\vec{\tau}$ and type τ' , $\text{INST}(\sigma, \vec{\tau}) = \tau' \implies \emptyset, \emptyset \vdash_{\mathcal{O}} v : \tau'$, then $\Gamma, \Delta \vdash_{\mathcal{O}} e[x := v] : \tau$.*

This lemma establishes that type safety is preserved when substituting a value into an expression, including for variables associated with polymorphic schemas. Its proof is carried out by recurrence on the structure of e .

Lemma 5.5 (Typing Preservation under Overload Substitution). *Let $\Gamma, \Delta, f, \sigma, e, v, \tau$ be given. If $\Gamma, \Delta \cup \{f : \sigma\} \vdash_{\mathcal{O}} e : \tau$, and if for every vector $\vec{\tau}$ and type τ' , $\text{INST}(\sigma, \vec{\tau}) = \tau' \implies \emptyset, \emptyset \vdash_{\mathcal{O}} v : \tau'$, then $\Gamma, \Delta \vdash_{\mathcal{O}} e[[f := v]] : \tau$.*

The proof of this lemma is analogous to the proof of the substitution on Γ , adapted to instances of extensions **ext** $f[\vec{\tau}]$.

Lemma 5.6 (Uniqueness of scheme instantiation). *The instantiation application INST is functional: for any scheme σ and any type vector $\vec{\tau}$,*

$$\text{INST}(\sigma, \vec{\tau}) = \tau_1 \wedge \text{INST}(\sigma, \vec{\tau}) = \tau_2 \implies \tau_1 = \tau_2$$

Theorem 5.7 (Type preservation). *Let e, e' be two terms and τ a type. If $\emptyset, \emptyset \vdash_{\mathcal{O}} e : \tau$ and if $e \longrightarrow e'$, then $\emptyset, \emptyset \vdash_{\mathcal{O}} e' : \tau$.*

This proof shows that the reduction of a properly typed term preserves its type entirely. This theorem is the second important theorem for proving the soundness of our typing system. To prove it, we generalize τ and proceed by recurrence on the structure of $e \longrightarrow e'$. The remaining cases to handle are trivial, with a few caveats necessary for cases involving variable substitution. Thus, we use the lemma on type preservation via substitution (Lemma 5.4) as well as the lemma on type preservation for overloaded functions via substitution (Lemma 5.5).

Lemma 5.8 (Substitution Preserves Property-Freeness). *Let x be a variable, and let v and e be terms. If e has no properties and v has no properties either, then $e[x := v]$ has no properties.*

For this lemma, we will reason by recurrence on the structure e . We will not introduce the proposition “ e is free of properties” in order to generalize it implicitly during the recurrence. Next, the reasoning proceeds by case, specifically by decomposing the hypothesis H_e into several subterms $e_1, e_2, \dots$.

Lemma 5.9 (Step reduction preserves property-freeness). *Let e and e' be two terms. If $e \longrightarrow e'$ and e has no properties, then e' has no properties.*

This final lemma before the main proposition states that if e does indeed reduce to e' and e was free of properties, then e' remains free of properties. The proof is carried out by recurrence on the structure of $e \longrightarrow e'$. Most cases are trivial to handle, and the few that do not resolve automatically simply require applying the property-free substitution theorem (Lemma 5.8).

5.2.2. The Soundness Theorem. Now, before proceeding with the proof of the soundness theorem, we must first define the concept of blocking for a term. A term is blocked if there is no term t' such that $t \longrightarrow t'$, and t is not a value. In other words, this amounts to saying that:

Definition 5.10 (Stuck term). A term is blocked if there is no term t' such that $t \longrightarrow t'$, and t is not a value: $\neg(\exists t'. t \longrightarrow t') \wedge \neg(\text{value } t)$

Theorem 5.11 (Type System Soundness). *Let e and e' be two terms, and τ a type. If e is free of properties (H_{free}), and e is well-typed with type τ in two empty environments (H_{type}), and finally $e \longrightarrow^* e'$, then e' is not blocked.*

The proof of the soundness theorem is actually not that complex, thanks to the many lemmas we have defined and proven. Thus, to prove this theorem, we proceed by recurrence on the multi-step reduction. We also generalize τ , H_{free} , and H_{type} .

In the first case, we seek to obtain **value** e using the available assumptions. Next, we apply the progression theorem (Theorem 5.3) to obtain two subgoals: **value** e and $e \longrightarrow e'$. For the first subgoal, we have the exact proposition in our assumptions. For the second subgoal, we use the normal form $(\forall R t. \neg(\exists t', R t t'))$ to obtain an existential construction. We must choose a t' such that $e \longrightarrow t'$; it turns out that e' corresponds to this construction. We conclude by the presence of the same proposition in the assumptions.

For the second case, we obtain the property-freeness of y thanks to the lemma on the preservation of property-freeness by reduction (Lemma 5.9) applied to $x \longrightarrow y$ and the property-freeness of x . Similarly, we seek to establish that y is well-typed as type τ in two empty environments using the type preservation by reduction (Theorem 5.7) applied to $x \longrightarrow y$ and to H_{type} . It then suffices to apply the recurrence hypothesis, assuming the property-freeness of y and the correct typing of y . Next, we destroy **stuck** z in two substeps: **normal** $(\longrightarrow) z$ and $\neg(\text{value } z)$. These two substeps are trivial since the propositions in question are present in the assumptions.

5.2.3. Uniqueness of Types. That said, we still have one final property of our type system to prove: the uniqueness of types in our system. By uniqueness, we mean that for the same expression and the same environments, but with two arbitrary types, the two types must necessarily be equivalent.

Lemma 5.12 (Variable Lookup Uniqueness). *Let $\Gamma, x, \tau_1, \tau_2$ be two arbitrary types, and let Γ be an environment. If $x : \tau_1 \in \Gamma$ and $x : \tau_2 \in \Gamma$, then $\tau_1 = \tau_2$.*

The proof essentially relies on the recurrence in the first lookup, with the second lookup assumption generalized. We conclude the lemma either by contradiction or by the presence of the same assumption.

Lemma 5.13 (Property Lookup Uniqueness). *Let $\Delta, n, \sigma_1, \sigma_2$ be two arbitrary types, and let Δ be an environment. If $n : \sigma_1 \in \Delta$ and $n : \sigma_2 \in \Delta$, then $\sigma_1 = \sigma_2$.*

This time, the proof is also relatively trivial; it suffices to note that $\Delta x = \sigma_1$ and $\Delta x = \sigma_2$ can be rewritten such that $\sigma_1 = \sigma_2$. This, of course, completes the proof.

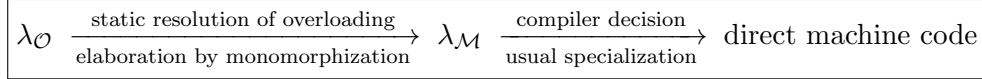
Theorem 5.14 (Type Uniqueness). *Let $\Gamma, \Delta, e, \tau_1, \tau_2$ be two arbitrary types, and let Γ, Δ be environments. If $\Gamma, \Delta \vdash e : \tau_1$ and $\Gamma, \Delta \vdash e : \tau_2$, then $\tau_1 = \tau_2$.*

As for this proof, we proceed as usual by recurrence on the correct typing of $e : \tau_1$. Once again, the goal is to generalize the correct typing of $e : \tau_2$. The proofs then rely primarily on the lemmas of uniqueness of instantiation (Lemma 5.6), uniqueness of property lookup (Lemma 5.13), and uniqueness of variable lookup (Lemma 5.12).

6. MONOMORPHIZATION

Now that we have proven the correctness of the type system of our source language $\lambda_{\mathcal{O}}$, we establish the safety of the target calculus $\lambda_{\mathcal{M}}$ —free of ad-hoc polymorphism—the semantic consistency of the construction via a forward simulation theorem, the algebraic properties of the specificity preorder, and the determinism of selection under the condition of well-formed catalog (WFC).

The overall architecture of our formalization and the compilation chain is summarized in the following conceptual diagram:



But first and foremost, we define the syntax of our target language $\lambda_{\mathcal{M}}$ and its type system.

6.1. Monomorphized Language Syntax. This syntax omits extension and property constructions. In the monomorphization pass, these are completely eliminated. Let us clarify an essential terminological distinction here: in this work, the term "monomorphization" specifically refers to the static resolution and specialization of *ad-hoc polymorphism* (i.e., the elimination of extension declarations, properties, and overloaded calls **ext** $n[\tau^k]$ in favor of target terms e_m). Parametric polymorphism of the Let-polymorphism type (**let** $x (\alpha^k) : \tau = e_1$ **in** e_2) is preserved in the target calculus $\lambda_{\mathcal{M}}$ to ensure compositionality and the invariance of lexical bindings, without imposing forced inlining of pure generic functions.

Type	τ	$::=$	nat	
			bool	
			string	
			$\tau_1 \rightarrow \tau_2$	function type
			$\tau_1 \times \tau_2$	product type
			α	type variable
Expr	e	$::=$	$x[\tau^k]$	variable
			v	literal value
			$\lambda[x : \tau].e$	lambda abstraction
			$e_1 \ e_2$	function application
			$e_1 + e_2$	addition
			let $x (\alpha^k) : \tau = e_1$ in e_2	let binding
			(e_1, e_2)	product construction
			$\pi_1 e$	first projection
			$\pi_2 e$	second projection
			if e_1 then e_2 else e_3	conditional
Value	v	$::=$	0	
			true	
			"Hello world"	
			...	

Figure 5: $\lambda_{\mathcal{M}}$'s Language Syntax

Thus, the target language $\lambda_{\mathcal{M}}$ is a standard functional calculus without residual overloading, free of any dictionary or dynamic resolution mechanism. This greatly simplifies safety and forward simulation proofs while ensuring direct compilation.

$$\begin{array}{c}
 \frac{x : \sigma \in \Gamma_m \quad \text{INST}(\sigma, \vec{\tau}) = \tau}{\Gamma_m \vdash_{\mathcal{M}} x[\vec{\tau}] : \tau} \text{ TM-VAR} \qquad \frac{}{\Gamma_m \vdash_{\mathcal{M}} n : \mathbf{nat}} \text{ TM-NUM} \\
 \\
 \frac{}{\Gamma_m \vdash_{\mathcal{M}} s : \mathbf{string}} \text{ TM-STRING} \qquad \frac{}{\Gamma_m \vdash_{\mathcal{M}} b : \mathbf{bool}} \text{ TM-BOOL} \\
 \\
 \frac{\Gamma_m \cup \{x : \tau_1\} \vdash_{\mathcal{M}} e : \tau_2}{\Gamma_m \vdash_{\mathcal{M}} \lambda[x : \tau_1].e : \tau_1 \rightarrow \tau_2} \text{ TM-ABS} \qquad \frac{\Gamma_m \vdash_{\mathcal{M}} e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma_m \vdash_{\mathcal{M}} e_2 : \tau_1}{\Gamma_m \vdash_{\mathcal{M}} e_1 \ e_2 : \tau_2} \text{ TM-APP} \\
 \\
 \frac{\Gamma_m \vdash_{\mathcal{M}} e_1 : \mathbf{nat} \quad \Gamma_m \vdash_{\mathcal{M}} e_2 : \mathbf{nat}}{\Gamma_m \vdash_{\mathcal{M}} e_1 + e_2 : \mathbf{nat}} \text{ TM-ADD} \\
 \\
 \frac{\Gamma_m \vdash_{\mathcal{M}} e_1 : \mathbf{bool} \quad \Gamma_m \vdash_{\mathcal{M}} e_2 : \tau \quad \Gamma_m \vdash_{\mathcal{M}} e_3 : \tau}{\Gamma_m \vdash_{\mathcal{M}} \mathbf{if} \ e_1 \ \mathbf{then} \ e_2 \ \mathbf{else} \ e_3 : \tau} \text{ TM-IF} \\
 \\
 \frac{\Gamma_m \vdash_{\mathcal{M}} e_1 : \tau_1 \quad \Gamma_m \vdash_{\mathcal{M}} e_2 : \tau_2}{\Gamma_m \vdash_{\mathcal{M}} (e_1, e_2) : \tau_1 \times \tau_2} \text{ TM-PAIR} \qquad \frac{\Gamma_m \vdash_{\mathcal{M}} e : \tau_1 \times \tau_2}{\Gamma_m \vdash_{\mathcal{M}} \pi_1 \ e : \tau_1} \text{ TM-FIRST} \\
 \\
 \frac{\Gamma_m \vdash_{\mathcal{M}} e : \tau_1 \times \tau_2}{\Gamma_m \vdash_{\mathcal{M}} \pi_2 \ e : \tau_2} \text{ TM-SECOND} \\
 \\
 \frac{(\forall \vec{\tau}. \text{INST}(\forall \vec{\alpha}. \tau_1, \vec{\tau}) = \tau' \implies \Gamma_m \vdash_{\mathcal{M}} e_1 : \tau') \quad \Gamma_m \cup \{x : \forall \vec{\alpha}. \tau_1\} \vdash_{\mathcal{M}} e_2 : \tau_2}{\Gamma_m \vdash_{\mathcal{M}} \mathbf{let} \ x[\vec{\alpha}] = e_1 \ \mathbf{in} \ e_2 : \tau_2} \text{ TM-LET}
 \end{array}$$

 Figure 6: Type system rules for the language $\lambda_{\mathcal{M}}$

6.2. Type System. The context of properties Δ has disappeared, leaving only the single context of variables Γ_m . This choice is entirely consistent, since the goal of monomorphization is to eliminate all traces of properties and ad-hoc polymorphism, which until then existed only in the context Δ .

$$\begin{array}{c}
\frac{\text{value}_{\mathcal{M}} v}{(\lambda(x : \tau). e) v \longrightarrow_{\mathcal{M}} e[x := v]} \text{STM-APPABS} \qquad \frac{e_1 \longrightarrow_{\mathcal{M}} e'_1}{(e_1 e_2) \longrightarrow_{\mathcal{M}} (e'_1 e_2)} \text{STM-APP1} \\
\\
\frac{\text{value}_{\mathcal{M}} e_1 \quad e_2 \longrightarrow_{\mathcal{M}} e'_2}{(e_1 e_2) \longrightarrow_{\mathcal{M}} (e_1 e'_2)} \text{STM-APP2} \\
\\
\frac{n_1, n_2, n_3 \in \mathbb{N} \quad n_1 + n_2 = n_3}{n_1 + n_2 \longrightarrow_{\mathcal{M}} n_3} \text{STM-ADDCONST} \qquad \frac{e_1 \longrightarrow_{\mathcal{M}} e'_1}{e_1 + e_2 \longrightarrow_{\mathcal{M}} e'_1 + e_2} \text{STM-ADD1} \\
\\
\frac{\text{value}_{\mathcal{M}} e_1 \quad e_2 \longrightarrow_{\mathcal{M}} e'_2}{e_1 + e_2 \longrightarrow_{\mathcal{M}} e_1 + e'_2} \text{STM-ADD2} \\
\\
\frac{e_1 \longrightarrow_{\mathcal{M}} e'_1}{\text{let } x[\alpha^k] = e_1 \text{ in } e_2 \longrightarrow_{\mathcal{M}} \text{let } x[\alpha^k] = e'_1 \text{ in } e_2} \text{STM-LET} \\
\\
\frac{\text{value}_{\mathcal{M}} v}{\text{let } x[\alpha^k] = v \text{ in } e_2 \longrightarrow_{\mathcal{M}} e_2[x := v]} \text{STM-LETVALUE} \\
\\
\frac{}{\text{if true then } e_2 \text{ else } e_3 \longrightarrow_{\mathcal{M}} e_2} \text{STM-IFTRUE} \\
\\
\frac{}{\text{if false then } e_2 \text{ else } e_3 \longrightarrow_{\mathcal{M}} e_3} \text{STM-IFFALSE} \\
\\
\frac{e_1 \longrightarrow_{\mathcal{M}} e'_1}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \longrightarrow_{\mathcal{M}} \text{if } e'_1 \text{ then } e_2 \text{ else } e_3} \text{STM-IF} \\
\\
\frac{e_1 \longrightarrow_{\mathcal{M}} e'_1}{(e_1, e_2) \longrightarrow_{\mathcal{M}} (e'_1, e_2)} \text{STM-PAIR1} \qquad \frac{\text{value}_{\mathcal{M}} e_1 \quad e_2 \longrightarrow_{\mathcal{M}} e'_2}{(e_1, e_2) \longrightarrow_{\mathcal{M}} (e_1, e'_2)} \text{STM-PAIR2} \\
\\
\frac{\text{value}_{\mathcal{M}} v_1 \quad \text{value}_{\mathcal{M}} v_2}{\pi_1(v_1, v_2) \longrightarrow_{\mathcal{M}} v_1} \text{STM-FSTPAIR} \qquad \frac{e \longrightarrow_{\mathcal{M}} e'}{\pi_1 e \longrightarrow_{\mathcal{M}} \pi_1 e'} \text{STM-FST} \\
\\
\frac{\text{value}_{\mathcal{M}} v_1 \quad \text{value}_{\mathcal{M}} v_2}{\pi_2(v_1, v_2) \longrightarrow_{\mathcal{M}} v_2} \text{STM-SNDPAIR} \qquad \frac{e \longrightarrow_{\mathcal{M}} e'}{\pi_2 e \longrightarrow_{\mathcal{M}} \pi_2 e'} \text{STM-SND}
\end{array}$$

Figure 7: Reduction rules for the language $\lambda_{\mathcal{M}}$

6.3. Neutral form and Reduction rules. As can be seen, these rules are essentially identical to those already defined for the syntax of $\lambda_{\mathcal{O}}$ (Figure 3). The rules do not differ for the sole reason that our syntax in terms of semantic reductions is identical to that of $\lambda_{\mathcal{O}}$. Indeed, since properties and extensions are not handled by $\lambda_{\mathcal{O}}$ in the operational semantics, our syntax is therefore fully compatible with that of $\lambda_{\mathcal{O}}$.

Now, we can also define the values of the language, which, unsurprisingly, are the same as those of $\lambda_{\mathcal{O}}$.

$$\begin{array}{c}
 \frac{n \in \mathbb{N}}{\mathbf{value}_{\mathcal{M}} n} \text{ VM-CONST} \qquad \frac{b \in \{\mathbf{true}, \mathbf{false}\}}{\mathbf{value}_{\mathcal{M}} b} \text{ VM-BOOL} \qquad \frac{s \in \mathbb{S}}{\mathbf{value}_{\mathcal{M}} s} \text{ VM-STRING} \\
 \\
 \frac{}{\mathbf{value}_{\mathcal{M}} \lambda(x : \tau).e} \text{ VM-ABS} \qquad \frac{\mathbf{value}_{\mathcal{M}} e_1 \quad \mathbf{value}_{\mathcal{M}} e_2}{\mathbf{value}_{\mathcal{M}} (e_1, e_2)} \text{ VM-PAIR}
 \end{array}$$

Figure 8: Reduction rules for the language $\lambda_{\mathcal{M}}$

6.4. Inductive Monomorphization Definition. We will now define, in the manner of a property, the monomorphism of a term $\lambda_{\mathcal{O}}$ to $\lambda_{\mathcal{M}}$ with an environment Ω . This environment Ω is constructed from a list of variable bindings, their instantiation types, and the monomorphized expression.

$$\begin{array}{c}
 \frac{n \in \mathbb{N}}{\Omega \vdash n \hookrightarrow n} \text{ ME-CONST} \qquad \frac{b \in \{\mathbf{true}, \mathbf{false}\}}{\Omega \vdash b \hookrightarrow b} \text{ ME-BOOL} \qquad \frac{s \in \mathbb{S}}{\Omega \vdash s \hookrightarrow s} \text{ ME-STRING} \\
 \\
 \frac{}{\Omega \vdash x \hookrightarrow x} \text{ ME-VAR} \qquad \frac{(n, \tau^k, e) \in \Omega}{\Omega \vdash \mathbf{ext } n[\tau^k] \hookrightarrow e} \text{ ME-OVER} \\
 \\
 \frac{\Omega \vdash e_1 \hookrightarrow e'_1 \quad \Omega \vdash e_2 \hookrightarrow e'_2}{\Omega \vdash e_1 e_2 \hookrightarrow e'_1 e'_2} \text{ ME-APP} \qquad \frac{\Omega \vdash e_1 \hookrightarrow e'_1 \quad \Omega \vdash e_2 \hookrightarrow e'_2}{\Omega \vdash e_1 + e_2 \hookrightarrow e'_1 + e'_2} \text{ ME-ADD} \\
 \\
 \frac{\Omega \vdash e \hookrightarrow e'}{\Omega \vdash \lambda(x : \tau).e \hookrightarrow \lambda(x : \tau).e'} \text{ ME-ABS} \\
 \\
 \frac{\Omega \vdash e_1 \hookrightarrow e'_1 \quad \Omega \vdash e_2 \hookrightarrow e'_2}{\Omega \vdash \mathbf{let } x[\alpha^k] = e_1 \mathbf{in } e_2 \hookrightarrow \mathbf{let } x[\alpha^k] = e'_1 \mathbf{in } e'_2} \text{ ME-LET} \\
 \\
 \frac{\Omega \vdash e \hookrightarrow e'}{\Omega \vdash \mathbf{prop } n(\alpha^k) : \tau \mathbf{in } e \hookrightarrow e'} \text{ ME-PROPERTY} \qquad \frac{\Omega \vdash e_1 \hookrightarrow e'_1 \quad \Omega \vdash e_2 \hookrightarrow e'_2}{\Omega \vdash (e_1, e_2) \hookrightarrow (e'_1, e'_2)} \text{ ME-PAIR} \\
 \\
 \frac{\Omega \vdash e \hookrightarrow e'}{\Omega \vdash \pi_1 e \hookrightarrow \pi_1 e'} \text{ ME-FST} \qquad \frac{\Omega \vdash e \hookrightarrow e'}{\Omega \vdash \pi_2 e \hookrightarrow \pi_2 e'} \text{ ME-SND} \\
 \\
 \frac{\Omega \vdash e_1 \hookrightarrow e'_1 \quad \Omega \vdash e_2 \hookrightarrow e'_2 \quad \Omega \vdash e_3 \hookrightarrow e'_3}{\Omega \vdash \mathbf{if } e_1 \mathbf{then } e_2 \mathbf{else } e_3 \hookrightarrow \mathbf{if } e'_1 \mathbf{then } e'_2 \mathbf{else } e'_3} \text{ ME-IF}
 \end{array}$$

Figure 9: Monomorphization rules for $\lambda_{\mathcal{M}}$

From now on, we can define certain concepts related to the monomorphization pass. In particular, we can define the validity of the choice of properties and extensions, the most specific choice of extensions, the validity of the monomorphization, the set of extension variables, the selection of an extension, the concept of coverage of Ω , and also the coverage of the extension catalog.

The concrete difference between the monomorphization instance table and the global catalog lies in the nature of the recorded types: the catalog C contains the implementation declarations created at the global level, whereas Ω represents the specialization table that is concretely instantiated for the generation of monomorphized code.

To illustrate this difference, let's consider an example: before monomorphization, implementation declarations are collected in the following form: $(f, \forall \vec{\alpha}. \tau, e_m)$. Thus, for two implementations of the overloaded symbol f of type $\forall \alpha_1 \alpha_2. (\alpha_1, \alpha_2)$ and of type $\forall. \mathbf{nat}$, the catalog C records two distinct entries for f :

$$(f, \forall \alpha_1 \alpha_2. (\alpha_1, \alpha_2), e_{f_tuple}) \quad (6.1)$$

$$(f, \forall. \mathbf{nat}, e_{f_nat}) \quad (6.2)$$

During monomorphization, the specialization table Ω is used to associate each concrete call site with the implementation determined based on the types of the arguments received. Formally, Ω is modeled as a finite set of triples $\Omega \subseteq \mathbf{PropName} \times \mathbf{Type}^k \times \mathbf{Expr}_{\mathcal{M}}$. Under the consistency condition $\mathbf{WFC}(C)$ (Definition 6.10), Ω is interpreted as a partial functional relation that associates a unique target implementation e_m with each pair $(f, \vec{\tau})$: $(f, \vec{\tau}, e_1) \in \Omega \wedge (f, \vec{\tau}, e_2) \in \Omega \implies e_1 = e_2$. Thus, for the previous example, the table Ω may contain:

$$\begin{aligned} \Omega = \{ & (f, (\mathbf{nat}), e_{f_nat}), \\ & (f, (\mathbf{nat}, \mathbf{nat}), e_{f_tuple_nat_nat}), \\ & (f, (\mathbf{string}, \mathbf{string}), e_{f_tuple_string_string}) \} \end{aligned} \quad (6.3)$$

To clarify the interactions between the various context structures involved in meta-theory, Table 2 summarizes the formal notations used for the source language $\lambda_{\mathcal{O}}$, the construction, and the target language $\lambda_{\mathcal{M}}$.

Symbol	Formal Role	Domain / Elements
Γ	Source Lexical Variables Context	$\{(x, \sigma)\}$ (typed variables of $\lambda_{\mathcal{O}}$)
Δ	Overload Signature Context	$f \mapsto \sigma$ (properties / general schemas)
C	Global Extension Catalog	$\{(f, \sigma, e_m)\}$ (implementation declarations)
Ω	Monomorphization Instance Table	$\{(f, \vec{\tau}, e_m)\}$ (target monomorphized specializations)
Γ_m	Target Lexical Variables Context	$\{(x, \sigma)\}$ (target language variables)

Table 2: Summary of Contexts and Environments in the System

Conceptual link between Δ , C and Ω . It is important to clarify the complementary roles and respective levels of abstraction of Δ , C , and Ω in the resolution process:

- Δ governs the lexical scope and local type contract of the properties accessible at the call site. According to the source typing rule T-Over (Figure 2), an overloaded call expression

ext $f[\vec{\tau}]$ can appear in a well-typed term only if the property f is declared in the local context Δ ($f : \sigma_\Delta \in \Delta$).

- C is the global catalog of candidate implementations declared throughout the program. This catalog may well contain concrete specializations that do not appear literally in Δ : for example, if Δ declares the abstract interface signature $\text{Eq} : \forall \alpha. \alpha \rightarrow \alpha \rightarrow \text{bool}$, the catalog C includes concrete specializations such as $(\text{Eq}, \text{nat} \rightarrow \text{nat} \rightarrow \text{bool}, e_{\text{nat}})$, $(\text{Eq}, \text{string} \rightarrow \text{string} \rightarrow \text{bool}, e_{\text{string}})$, or even implementations for other properties outside the lexical scope of Δ . It would therefore be incorrect to require $(f, \sigma_C, e) \in \Delta$, since Δ contains only the abstract declarations of the properties.
- Ω represents the actual result of the construction: it is the canonical specialization environment resulting from the static selection (**SELECT**) performed on C , associating with each pair $(f, \vec{\tau})$ appearing in the overloaded calls the unique resolved specialized implementation.

In order to formally link the global catalog C to the local context Δ , we introduce a *conformity* relation: every implementation present in C for a symbol declared in Δ must be a specialization compatible with the abstract signature prescribed by Δ .

Definition 6.1 (Type Scheme Compatibility). Two type schemas σ_Δ and σ_C are said to be compatible if any joint instantiation over the same tuple of concrete argument types $\vec{\tau}$ produces the same instantiated type:

$$\text{COMPATIBLE}(\sigma_\Delta, \sigma_C) \stackrel{\text{def}}{=} \forall \vec{\tau} \tau_\Delta \tau_C. (\text{INST}(\sigma_\Delta, \vec{\tau}) = \tau_\Delta \wedge \text{INST}(\sigma_C, \vec{\tau}) = \tau_C) \implies \tau_\Delta = \tau_C$$

Definition 6.2 (Catalog Conformance to Δ). A catalog of extensions C conforms to a context of properties Δ (denoted $\text{CONFORMS}(C, \Delta)$ or $C \models \Delta$) if every entry in C whose symbol is declared in Δ has a schema compatible with the signature of Δ :

$$\text{CONFORMS}(C, \Delta) \stackrel{\text{def}}{=} \forall (f, \sigma_C, e_m) \in C, \forall \sigma_\Delta. (f : \sigma_\Delta \in \Delta) \implies \text{COMPATIBLE}(\sigma_\Delta, \sigma_C)$$

This definition applies only to entries in C whose symbol appears in Δ , thereby preserving the open and modular nature of the global catalog.

Definition 6.3 (Extension Environment Match). An extension environment Ω matches a property context Δ (denoted $\text{MATCHES}(\Delta, \Omega)$) if every specialization recorded in Ω for a property declared in Δ is well-typed in the target calculus with the corresponding instantiated type. Formally:

$$\text{MATCHES}(\Delta, \Omega) \stackrel{\text{def}}{=} \forall f \sigma \vec{\tau} \tau e_m. f : \sigma \in \Delta \wedge \text{INST}(\sigma, \vec{\tau}) = \tau \wedge (f, \vec{\tau}, e_m) \in \Omega \implies \emptyset \vdash_{\mathcal{M}} e_m : \tau$$

Lemma 6.4 (Conformity and Validity Implies Correspondence). *For any catalog C , any property context Δ , and any instance table Ω , if C conforms to Δ and if Ω is a valid monomorphism of C , then Ω corresponds to Δ :*

$$\text{CONFORMS}(C, \Delta) \wedge \text{VALID}(C, \Omega) \implies \text{MATCHES}(\Delta, \Omega)$$

Proof. Let $f, \sigma_\Delta, \vec{\tau}, \tau_\Delta, e_m$ be such that $f : \sigma_\Delta \in \Delta$, $\text{INST}(\sigma_\Delta, \vec{\tau}) = \tau_\Delta$, and $(f, \vec{\tau}, e_m) \in \Omega$. By the assumption of monomorphism validity ($\text{VALID}(C, \Omega)$), the entry $(f, \vec{\tau}, e_m) \in \Omega$ must come from an entry selected from C : therefore, there exists a schema σ_C and a type τ_C such that $(f, \sigma_C, e_m) \in C$, $\text{INST}(\sigma_C, \vec{\tau}) = \tau_C$, and $\emptyset \vdash_{\mathcal{M}} e_m : \tau_C$. Since $(f, \sigma_C, e_m) \in C$ and $f : \sigma_\Delta \in \Delta$, the catalog conformity ($\text{CONFORMS}(C, \Delta)$) ensures that $\text{COMPATIBLE}(\sigma_\Delta, \sigma_C)$. Since $\text{INST}(\sigma_\Delta, \vec{\tau}) = \tau_\Delta$ and $\text{INST}(\sigma_C, \vec{\tau}) = \tau_C$, the definition of compatibility immediately implies $\tau_\Delta = \tau_C$. Consequently, $\emptyset \vdash_{\mathcal{M}} e_m : \tau_\Delta$, which concludes the proof of $\text{MATCHES}(\Delta, \Omega)$ (formalized under the name `catalog_conforms_implies_matches` in `Definitions.v`). $\square$

Definition 6.5 (More Specific Type Relation). A specificity relation (denoted $\tau_1 \preceq \tau_2$ or $\tau_1 \leq \tau_2$) is the instantiation preorder between types: a type τ_1 is more specific than a type τ_2 if it constitutes a structural specialization of τ_2 (where variables of type α are the most general elements). Note that this is indeed a **preorder** (reflexive and transitive) and not a partial order: in fact, for two variables of distinct types $\alpha \neq \beta$, the rule MS-VAR allows $\alpha \preceq \beta$ and $\beta \preceq \alpha$, so that $\alpha \equiv \beta$. The associated strict relation $\prec$ is the strict part of this preorder (anti-reflexive and transitive); the uniqueness of the schemas is therefore considered modulo the equivalence induced by the specificity preorder.

$$\begin{array}{c}
\frac{}{\tau \preceq \tau} \text{MS-REFL} \qquad \frac{}{\tau \preceq \alpha} \text{MS-VAR} \qquad \frac{\tau_1 \preceq \tau'_1 \quad \tau_2 \preceq \tau'_2}{\tau_1 \rightarrow \tau_2 \preceq \tau'_1 \rightarrow \tau'_2} \text{MS-ARROW} \\
\\
\frac{\tau_1 \preceq \tau'_1 \quad \tau_2 \preceq \tau'_2}{(\tau_1, \tau_2) \preceq (\tau'_1, \tau'_2)} \text{MS-TUPLE} \qquad \frac{}{\mathbf{nat} \preceq \mathbf{nat}} \text{MS-NAT} \qquad \frac{}{\mathbf{bool} \preceq \mathbf{bool}} \text{MS-BOOL} \\
\\
\frac{}{\mathbf{string} \preceq \mathbf{string}} \text{MS-STRING}
\end{array}$$

Figure 10: Specific type relation rules

We naturally extend this relation to type schemas: a schema $\sigma_1 = \forall \alpha^k. \tau_1$ is more specific than $\sigma_2 = \forall \beta^j. \tau_2$ if the underlying type τ_1 is more specific than τ_2 . Comparisons between schemas are made modulo α -renaming of bound variables (α -equivalence), so that the choice of names for quantifier variables does not affect the specificity relation (for example, $\forall \alpha. \alpha \rightarrow \alpha \equiv \forall \beta. \beta \rightarrow \beta$).

Definition 6.6 (Strictly More Specific Type Scheme Relation). A σ_1 -schema is strictly more specific than σ_2 (denoted $\sigma_1 \prec \sigma_2$) if $\sigma_1 \preceq \sigma_2$ and $\neg(\sigma_2 \preceq \sigma_1)$.

Definition 6.7 (Scheme Equivalence). Two schemas of types σ_1 and σ_2 are equivalent in specificity (denoted $\sigma_1 \equiv \sigma_2$) if they are mutually more specific than each other:

$$\sigma_1 \equiv \sigma_2 \stackrel{\text{def}}{=} \sigma_1 \preceq \sigma_2 \wedge \sigma_2 \preceq \sigma_1$$

Definition 6.8 (Scheme Candidate). A σ -type is a candidate for an overloaded symbol f and argument types $\vec{\tau}$ in the catalog C if and only if:

$$\text{SC}(C, f, \vec{\tau}, \sigma) \stackrel{\text{def}}{=} \exists e_m \tau. (f, \sigma, e_m) \in C \wedge \text{INST}(\sigma, \vec{\tau}) = \tau \wedge \emptyset \vdash_{\mathcal{M}} e_m : \tau$$

Definition 6.9 (Most Specific Candidate). Given a catalog of extensions C , an overloaded symbol f , and a tuple of instantiation types $\vec{\tau}$, a schema σ is the most specific candidate among all possible candidates in C if and only if:

$$\text{MSC}(C, f, \vec{\tau}, \sigma) \stackrel{\text{def}}{=} \text{SC}(C, f, \vec{\tau}, \sigma) \wedge (\forall \sigma'. \text{SC}(C, f, \vec{\tau}, \sigma') \implies \neg(\sigma' \prec \sigma))$$

Definition 6.10 (Coherent Catalog). A catalog of extensions C is said to be well-formed ($\text{WFC}(C)$) if, for every overloaded symbol f , every tuple of instantiation types $\vec{\tau}$, and every set of maximal candidates in C , the schemas are equivalent and the implementations coincide:

$$\text{WFC}(C) \stackrel{\text{def}}{=} \forall f \vec{\tau} \sigma_1 \sigma_2 e_1 e_2. (f, \sigma_1, e_1) \in C \wedge (f, \sigma_2, e_2) \in C$$

$$\begin{aligned} & \wedge \text{MSC}(C, f, \vec{\tau}, \sigma_1) \wedge \text{MSC}(C, f, \vec{\tau}, \sigma_2) \\ \implies & \sigma_1 \equiv \sigma_2 \wedge e_1 = e_2 \end{aligned}$$

This condition excludes situations in which multiple incompatible maximal candidates cover the same call point, as well as distinct implementations associated with equivalent maximal schemas.

Lemma 6.11 (Non-Strict Ordering of Most Specific Candidates). *Let C be a catalog of extensions, f an overloaded symbol, $\vec{\tau}$ a tuple of types, and σ_1, σ_2 two candidate schemas that can be instantiated over $\vec{\tau}$. If $\text{MSC}(C, f, \vec{\tau}, \sigma_1)$ and $\text{MSC}(C, f, \vec{\tau}, \sigma_2)$, then neither schema is strictly more specific than the other:*

$$\neg(\sigma_1 \prec \sigma_2) \wedge \neg(\sigma_2 \prec \sigma_1)$$

This structural result (formally proven in our Rocq/Coq archive under the name `most_specific_candidate_unique`) establishes that two candidates satisfying `MSC` cannot be strictly ordered.

Definition 6.12 (Monomorphization is Valid). Given a catalog of extensions C and a table of instances Ω , the monomorphism is valid ($\text{VALID}(C, \Omega)$) means that every tuple $(f, \vec{\tau}, e_m) \in \Omega$ originates from an entry declared in C , selected as the most specific candidate, and that e_m is well-typed in the target calculus with the instantiated type:

$$\begin{aligned} \text{VALID}(C, \Omega) \stackrel{\text{def}}{=} \forall f \vec{\tau} e_m. (f, \vec{\tau}, e_m) \in \Omega \implies \exists \sigma \tau. (f, \sigma, e_m) \in C \wedge \text{INST}(\sigma, \vec{\tau}) = \tau \\ \wedge \text{MSC}(C, f, \vec{\tau}, \sigma) \wedge \emptyset \vdash_{\mathcal{M}} e_m : \tau \end{aligned}$$

Definition 6.13 (Overloaded Variable Collection Getter). Given a source term e , let $\Phi(e)$ denote the list of overloaded calls $(f, \vec{\tau})$ present in e . It is defined inductively as follows:

$$\begin{aligned} \Phi(\mathbf{ext} \ f[\vec{\tau}]) &= [(f, \vec{\tau})] \\ \Phi(e_1 \ e_2) &= \Phi(e_1) ++ \Phi(e_2) \\ \Phi(e_1 + e_2) &= \Phi(e_1) ++ \Phi(e_2) \\ \Phi((e_1, e_2)) &= \Phi(e_1) ++ \Phi(e_2) \\ \Phi(\mathbf{let} \ _[\vec{\alpha}] = e_1 \ \mathbf{in} \ e_2) &= \Phi(e_1) ++ \Phi(e_2) \\ \Phi(\lambda(_ : _). e) &= \Phi(e) \\ \Phi(\mathbf{prop} \ _(\vec{\alpha}) : _ \mathbf{in} \ e) &= \Phi(e) \\ \Phi(\dots) &= [] \quad \text{for constants and lexical variables} \end{aligned}$$

Definition 6.14 (Applicable Extension Candidate). An overloaded call $(f, \vec{\tau})$ is said to be applicable in a catalog C (denoted $\text{Applicable}(C, f, \vec{\tau})$) if there exists at least one entry declared in C for f whose signature instantiates over $\vec{\tau}$ to a well-typed term in the target calculus:

$$\text{Applicable}(C, f, \vec{\tau}) \stackrel{\text{def}}{=} \exists \sigma \tau e_m. (f, \sigma, e_m) \in C \wedge \text{INST}(\sigma, \vec{\tau}) = \tau \wedge \emptyset \vdash_{\mathcal{M}} e_m : \tau$$

Lemma 6.15 (Existence of Most Specific Candidate). *For any finite catalog C , any symbol f , and any tuple of instantiation types $\vec{\tau}$, if the call is applicable in C , then there exists at least one most specific candidate in the sense of `MSC`:*

$$\text{Applicable}(C, f, \vec{\tau}) \implies \exists \sigma. \text{MSC}(C, f, \vec{\tau}, \sigma)$$

Proof. Let $\mathcal{S} = \{\sigma \mid \exists e_m \tau. (f, \sigma, e_m) \in C \wedge \text{INST}(\sigma, \vec{\tau}) = \tau \wedge \emptyset \vdash_{\mathcal{M}} e_m : \tau\}$. By assumption, $\mathcal{S}$ is nonempty. Since the catalog C is finite, $\mathcal{S}$ is a nonempty finite set. The relation $\prec$ is the strict part of a preorder (anti-reflexive and transitive, Coq lemmas `strictly_more_specific_scheme_irrefl` and `_trans`). Now, every irreflexive and transitive binary relation on a nonempty finite set has at least one maximal element $\sigma^* \in \mathcal{S}$ such that $\forall \sigma' \in \mathcal{S}, \neg(\sigma' \prec \sigma^*)$. By definition, such a scheme satisfies precisely $\text{MSC}(C, f, \vec{\tau}, \sigma^*)$. $\square$

Theorem 6.16 (Deterministic Implementation Selection under WFC). *For any finite catalog C , any overloaded symbol f , and any tuple $\vec{\tau}$, if the catalog is well-formed ($\text{WFC}(C)$) and if the call is applicable ($\text{Applicable}(C, f, \vec{\tau})$), then there exists a unique selectable specialized implementation e_m :*

$$\text{WFC}(C) \wedge \text{Applicable}(C, f, \vec{\tau}) \implies \exists! e_m. \text{SELECT}(C, f, \vec{\tau}, e_m)$$

Proof. The existence of e_m follows from the applicability of Lemma 6.15: there exists a maximal candidate $\sigma \in \mathcal{S}$ such that $(f, \sigma, e_m) \in C$, hence $\text{SELECT}(C, f, \vec{\tau}, e_m)$. For uniqueness, let e_1, e_2 be two terms satisfying $\text{SELECT}(C, f, \vec{\tau}, e_1)$ and $\text{SELECT}(C, f, \vec{\tau}, e_2)$. By the definition of SELECT , there exist schemas σ_1, σ_2 such that $(f, \sigma_i, e_i) \in C$ and $\text{MSC}(C, f, \vec{\tau}, \sigma_i)$ for $i \in \{1, 2\}$. The well-formedness condition $\text{WFC}(C)$ (Definition 6.10) then immediately implies $e_1 = e_2$. $\square$

Definition 6.17 (Extension Selection Through Catalog). The property $\text{SELECT}(C, f, \vec{\tau}, e_m)$ characterizes the selection of a specialized implementation e_m for an overloaded function f and argument types $\vec{\tau}$ from the catalog C :

$$\text{SELECT}(C, f, \vec{\tau}, e_m) \stackrel{\text{def}}{=} \exists \sigma \tau. (f, \sigma, e_m) \in C \wedge \text{INST}(\sigma, \vec{\tau}) = \tau \wedge \text{MSC}(C, f, \vec{\tau}, \sigma) \wedge \emptyset \vdash_{\mathcal{M}} e_m : \tau$$

Definition 6.18 (Extension Environment Covers Cases). This property ensures that every overloaded call present in a term e has a specialized entry resolved in Ω :

$$\text{COVERS}_{\Omega}(C, \Omega, e) \stackrel{\text{def}}{=} \text{VALID}(C, \Omega) \wedge \forall f \vec{\tau}. (f, \vec{\tau}) \in \Phi(e) \implies \exists e_m. (f, \vec{\tau}, e_m) \in \Omega$$

Definition 6.19 (Extension Catalog Covers Cases). This property guarantees that the extension catalog C is sufficient to resolve all overloaded calls to a term e :

$$\text{COVERS}(C, e) \stackrel{\text{def}}{=} \forall f \vec{\tau}. (f, \vec{\tau}) \in \Phi(e) \implies \exists e_m. \text{SELECT}(C, f, \vec{\tau}, e_m)$$

Modular Foundation of COVERS. The property $\text{COVERS}(C, e)$ requires that every call $(f, \vec{\tau}) \in \Phi(e)$ have a term selected via SELECT . By Lemma 6.15 and Theorem 6.16, the validity of this global assumption reduces to two local modular conditions:

- (1) *Local applicability:* for each call $(f, \vec{\tau}) \in \Phi(e)$, the catalog contains at least one instantiable instance ($\text{Applicable}(C, f, \vec{\tau})$), which ensures the existence of a resolved instance via Lemma 6.15;
- (2) *Global consistency:* the catalog is well-formed ($\text{WFC}(C)$), which eliminates ambiguities and guarantees the determinism of the selection.

Ω 's Determinism. Under the assumption of good formation $\text{WFC}(C)$, the induced partial application $(f, \vec{\tau}) \mapsto e_m$ defined by $\text{SELECT}(C, f, \vec{\tau}, e_m)$ is functional according to Theorem 6.16. Consequently, for a given closed set of calls $\beta = \Phi(e)$, the canonical environment Ω —which associates each pair $(f, \vec{\tau})$ appearing in the overloaded calls with its unique selected implementation—guarantees that the evaluation $\Omega \vdash e \hookrightarrow e_m$ produces a unique monomorphic term e_m .

Definition 6.20 (Weakening Relation on Contexts). Given two variable contexts Γ and Γ' , we define the context inclusion $\Gamma \subseteq \Gamma'$ as follows:

$$\forall x \sigma. \text{LOOKUP}(\Gamma, x, \sigma) \implies \text{LOOKUP}(\Gamma', x, \sigma)$$

Where $\text{LOOKUP}(\Gamma, x, \sigma)$ denotes the search for the variable x of schema type σ in Γ .

Lemma 6.21 (Context Inclusion Update). *Given two contexts Γ and Γ' , adding a variable $x : \sigma$ to Γ and Γ' preserves the inclusion $\Gamma \subseteq \Gamma'$.*

7. MONOMORPHIZATION PROOFS

Before beginning any proof of any theorem, we must define a few important lemmas in order to simplify the proofs. This is particularly true for lemmas of canonical form, as demonstrated in the previous sections.

Lemma 7.1 (Canonical Form). *Given a term e_m and a type nat (or bool , $\tau_1 \rightarrow \tau_2$, (τ_1, τ_2)), such that e_m is a monomorphized value in the sense of $\text{value}_{\mathcal{M}} e$, there exist terms, variables, and types such that $e_m = n$ (respectively, $e_m = b$, $e_m = \lambda(x : \tau_1)e$, $e_m = (e_1, e_2)$). In the case of an n -tuple, we will assume that the lemma also requires the presence of $\text{value}_{\mathcal{M}} e_1$ and $\text{value}_{\mathcal{M}} e_2$.*

The proofs of these lemmas are relatively trivial; we have already presented one way to prove them, so we will not present them here.

Lemma 7.2 (Context Inclusion Update). *Given two contexts Γ and Γ' of type $(\text{Var} \times \sigma)^*$, adding a variable $x : \sigma$ to both Γ and Γ' while preserving the inclusion also preserves the inclusion of Γ in Γ' .*

Proving this lemma amounts to performing a case analysis on the equality $x = x_0$ introduced by the context extension, and then applying the properties of the lookup relation LOOKUP .

The lemmas on context weakening and type preservation by substitution for the target calculus $\lambda_{\mathcal{M}}$ follow a structure analogous to the results established in Section 5 and are fully formalized in the Rocq implementation (`Proofs.v`).

7.1. Type System Soundness.

Theorem 7.3 (Progression). *For any closed term e_m and any type τ , if $\emptyset \vdash_{\mathcal{M}} e_m : \tau$, then either e_m is a value ($\text{value}_{\mathcal{M}} e_m$), or there exists a term e'_m such that $e_m \longrightarrow_{\mathcal{M}} e'_m$.*

Theorem 7.4 (Type preservation by reduction). *For all terms e_m, e'_m and any type τ , if $\emptyset \vdash_{\mathcal{M}} e_m : \tau$ and $e_m \longrightarrow_{\mathcal{M}} e'_m$, then $\emptyset \vdash_{\mathcal{M}} e'_m : \tau$.*

Theorem 7.5 (Type Soundness / Absence of Stuck States). *For all terms e_m, e'_m and any type τ , if $\emptyset \vdash_{\mathcal{M}} e_m : \tau$ and $e_m \longrightarrow^*_{\mathcal{M}} e'_m$, then e'_m is never stuck ($\neg \text{stuck}_{\mathcal{M}} e'_m$).*

These three theorems establish the fundamental execution safety properties of the target monomorphized calculus $\lambda_{\mathcal{M}}$. The detailed proofs are omitted here for the sake of brevity; they are fully formalized and mechanically verified without axioms in our Rocq/Coq implementation (`meta-theory/LangD/Proofs.v`, theorems `progress_m`, `preservation_m`, and `soundness_m`).

7.2. Monomorphization related Proofs. Now we can focus on the heart of the matter; in other words, we can prove the upcoming theorems concerning the consistency and soundness of monomorphism.

Lemma 7.6 (Monomorphization preserves Value). *For any extension environment Ω , any polymorphic term v , and any monomorphic term v_m , if $\text{value}_{\mathcal{M}} v$ and if $\Omega \vdash v \hookrightarrow v_m$, then $\text{value}_{\mathcal{M}} v_m$.*

This proof is carried out and demonstrated by recurrence on the monomorphism property; we then attempt, for each unresolved case, to reason about the structure of $\text{value } v$ using case-by-case analysis; we then substitute to finally conclude.

Lemma 7.7 (Context Matching implies Variable Lookup). *For all contexts Γ and Γ_m , and for any variable x of schema type σ , if $\text{MATCHES}(\Gamma, \Gamma_m)$ and $\text{LOOKUP}(\Gamma, x, \sigma)$, then $\text{LOOKUP}(\Gamma_m, x, \sigma)$.*

This property (mechanized in Rocq under the name `context_matches_lookup`) is proven by induction on the structure of $\text{MATCHES}(\Gamma, \Gamma_m)$, followed by a case-by-case analysis of the derivation of $\text{LOOKUP}(\Gamma, x, \sigma)$.

Lemma 7.8 (Extension Environment Matches Update). *For any property environment Δ , for any instance table Ω , and for any schema property f of the form $\forall \vec{\alpha}. \tau$, if $\text{MATCHES}(\Delta, \Omega)$ and if for all $\vec{\tau}, \tau', e_m$, $\text{INST}(\forall \vec{\alpha}. \tau, \vec{\tau}) = \tau'$ implies that $(f, \vec{\tau}, e_m) \in \Omega$ with $\emptyset \vdash_{\mathcal{M}} e_m : \tau'$, then $\text{MATCHES}(\{f : \forall \vec{\alpha}. \tau\} \cup \Delta, \Omega)$.*

The proof is carried out by case analysis based on the identity of the variable in question with respect to f , followed by the preservation of MATCHES when extending Δ .

Theorem 7.9 (Monomorphization Preserves Typing). *Let Γ and Δ be two source typing contexts, e a source term, and τ a type such that $\Gamma, \Delta \vdash_{\mathcal{O}} e : \tau$. For any catalog C , any instance table Ω , any target context Γ_m , and any target term e_m , if:*

- (1) $\text{MATCHES}(\Delta, \Omega)$;
- (2) $\text{MATCHES}(\Gamma, \Gamma_m)$;
- (3) $\text{VALID}(C, \Omega)$;
- (4) $\Omega \vdash e \hookrightarrow e_m$,

then $\Gamma_m \vdash_{\mathcal{M}} e_m : \tau$.

In the Rocq formalization (`Proofs.v`, theorem `monomorphization_preserves_typing`), the proof proceeds by induction on the type derivation $\Gamma, \Delta \vdash_{\mathcal{O}} e : \tau$, by universally quantifying over Ω, Γ_m, e_m , as well as on the environment compatibility hypotheses. The condition $\text{VALID}(C, \Omega)$ directly guarantees the invariant required by the recurrence, namely that every instance selected from Ω is well-typed in $\lambda_{\mathcal{M}}$ ($\forall f, \vec{\alpha}, \tau_0, \tau_1, \vec{\tau}, e'_m. \text{INST}(\forall \vec{\alpha}. \tau_0, \vec{\tau}) = \tau_1 \implies (f, \vec{\tau}, e'_m) \in \Omega \implies \emptyset \vdash_{\mathcal{M}} e'_m : \tau_1$). Each source typing case is resolved by case analysis on the construction relation $\Omega \vdash e \hookrightarrow e_m$, with the case of the local rule T-Prop relying on Lemma 7.8.

7.3. Semantic Preservation via Forward Simulation.

Definition 7.10 (Extension Environment is Closed). An extension environment Ω is closed if it contains no residual free variables in its implementation terms:

$$\text{CLOSED}(\Omega) \stackrel{\text{def}}{=} \forall (f, \vec{\tau}, e_m) \in \Omega, \text{FV}(e_m) = \emptyset$$

Lemma 7.11 (Closed Expressions are Invariant under Substitution). *For any target expression e_m , if $\text{FV}(e_m) = \emptyset$, then for any variable x and any value v , we have $e_m[x := v] = e_m$.*

To prove this lemma, we must first prove the sub-lemma stating that if $x \notin \text{FV}(e_m)$, then $e_m[x := v] = e_m$. The proof is relatively simple: we introduce a recurrence relation on e_m , consider the cases for the equality of variables introduced by the recurrence, and then conclude. Next, the proof of the original lemma is simply a special case of this sub-lemma.

Lemma 7.12 (Monomorphization preserves Substitution). *For any $\Omega, e, e_m, x, v, v_m$, if Ω is closed, and if $\Omega \vdash e \hookrightarrow e_m$, and $\Omega \vdash v \hookrightarrow v_m$, then $\Omega \vdash e[x := v] \hookrightarrow e_m[x := v_m]$.*

This proof uses the recurrence on $\Omega \vdash e \hookrightarrow e_m$ by generalizing the remaining properties that do not depend on this proposition. The rest of the proof consists of case distinctions for $x = x_0$ and $x \neq x_0$, and of the use of the closure property on Ω .

Lemma 7.13 (Step Simulation). *For any closed Ω , and for any terms x, y , and monomorphic term e_m , if $x \longrightarrow y$ and if $\Omega \vdash x \hookrightarrow e_m$, then there exists an e'_m such that $e_m \longrightarrow_{\mathcal{M}} e'_m$ and such that $\Omega \vdash y \hookrightarrow e'_m$.*

This lemma, formalized under the name `simulation_step` in Coq, states that if an elementary step is performed at the source level ($x \longrightarrow y$), the corresponding target term performs a simulation step ($e_m \longrightarrow_{\mathcal{M}} e'_m$) that preserves the monomorphism relation.

Theorem 7.14 (Forward Simulation Theorem). *For any closed Ω , and for any terms e, e' , if $e \longrightarrow^* e'$, then for any monomorphic term e_m , if $\Omega \vdash e \hookrightarrow e_m$, then there exists an e'_m such that $e_m \longrightarrow^*_{\mathcal{M}} e'_m$ and such that $\Omega \vdash e' \hookrightarrow e'_m$.*

This fundamental theorem (named `simulation_multi` in our Coq repository) establishes a forward simulation: every source reduction trace is faithfully reproduced by the target monomorphic term.

7.4. Monomorphization Existence and End-to-End Soundness. It is now possible to establish the existence property of the monomorphism, leading to the overall security of the system.

Lemma 7.15 (Monomorphized Term Existence). *For any extension catalog C , any extension environment Ω , and any expression e , if $\text{COVERS}_{\Omega}(C, \Omega, e)$, then there exists a monomorphic expression e_m such that $\Omega \vdash e \hookrightarrow e_m$.*

Lemma 7.16 (Omega Construction Using Calls). *For any catalog C and any list of overloaded calls β , if, for all $f, \vec{\tau}$, $(f, \vec{\tau}) \in \beta$ implies that there exists an e_m such that $\text{SELECT}(C, f, \vec{\tau}, e_m)$ is satisfied, then there exists an environment Ω such that $\text{VALID}(C, \Omega)$ and such that for every $(f, \vec{\tau}) \in \beta$, there exists an e_m such that $(f, \vec{\tau}, e_m) \in \Omega$.*

Lemma 7.17 (Omega Existence). *Let C be a catalog and e a term. If $\text{COVERS}(C, e)$, then there exists an environment Ω such that $\text{COVERS}_{\Omega}(C, \Omega, e)$.*

Theorem 7.18 (Monomorphization Existence). *Let C be a catalog and e be a term. If $\text{COVERS}(C, e)$, then there exists a monomorphic expression e_m and an environment Ω such that $\text{VALID}(C, \Omega)$ and $\Omega \vdash e \hookrightarrow e_m$.*

Theorem 7.19 (Monomorphization Soundness). *Let Γ, Δ be two source typing contexts, e a source term, τ a type, Ω an extension environment, e_m a target monomorphic term, Γ_m a target context, and C an extension catalog. If $\Gamma, \Delta \vdash_{\mathcal{O}} e : \tau$, $\text{COVERS}_{\Omega}(C, \Omega, e)$, $\text{MATCHES}(\Delta, \Omega)$, $\text{MATCHES}(\Gamma, \Gamma_m)$, $\text{VALID}(C, \Omega)$, and $\Omega \vdash e \hookrightarrow e_m$, then:*

$$\Gamma_m \vdash_{\mathcal{M}} e_m : \tau.$$

Theorem 7.20 (End-to-End Soundness). *Let e be a closed surface expression, properly typed under a property context Δ ($\emptyset, \Delta \vdash_{\mathcal{O}} e : \tau$), and let C be a catalog of instances satisfying the following conditions:*

- **(Coverage)** C covers all overloaded calls appearing in e ($\text{COVERS}(C, e)$);
 - **(Conformity)** C conforms to the context of the declared properties Δ ($\text{CONFORMS}(C, \Delta)$).
- Then, there exists a specialization environment Ω and a monomorphic target expression e_m such that:*

- (1) **(Validity and correspondence of the specialization)** $\text{VALID}(C, \Omega)$ and $\text{MATCHES}(\Delta, \Omega)$;
- (2) **(Success of derivation)** $\Omega \vdash e \hookrightarrow e_m$;
- (3) **(Preservation of typing)** $\emptyset \vdash_{\mathcal{M}} e_m : \tau$;
- (4) **(Dynamic execution safety)** For any term e'_m such that $e_m \rightarrow_{\mathcal{M}}^* e'_m$, the term e'_m is never stuck ($\neg \text{stuck}_{\mathcal{M}}(e'_m)$).

The demonstration constructively integrates the established modular results:

- (1) By the lemma on the existence of the specialization table (Lemma 7.17), the assumption $\text{COVERS}(C, e)$ provides an extension environment Ω such that $\text{VALID}(C, \Omega)$ and $\text{COVERS}_{\Omega}(C, \Omega, e)$ hold.
- (2) Since $\text{CONFORMS}(C, \Delta)$ and $\text{VALID}(C, \Omega)$, Lemma 6.4 applies directly and guarantees that Ω faithfully corresponds to Δ , i.e., $\text{MATCHES}(\Delta, \Omega)$. This deduction eliminates any need to rely on an arbitrary hypothesis universally quantified over the possible Ω 's.
- (3) By Lemma 7.15 (existence of monomorphic terms), condition $\text{COVERS}_{\Omega}(C, \Omega, e)$ ensures the existence of a monomorphic expression e_m such that $\Omega \vdash e \hookrightarrow e_m$.
- (4) The monomorphization soundness theorem (Theorem 7.19) then holds under the assumptions $\emptyset, \Delta \vdash_{\mathcal{O}} e : \tau$, $\text{COVERS}_{\Omega}(C, \Omega, e)$, $\text{MATCHES}(\Delta, \Omega)$, $\text{VALID}(C, \Omega)$, and $\text{MATCHES}(\emptyset, \emptyset)$, establishing the preservation of typing in the target calculus: $\emptyset \vdash_{\mathcal{M}} e_m : \tau$.
- (5) Finally, the semantic safety theorem of $\lambda_{\mathcal{M}}$ (`soundness_m`) guarantees that the well-typed closed target term e_m never evaluates to a blocked state.

The entire theorem is formalized under the name `end_to_end_soundness` in `Proofs.v`.

8. DISCUSSIONS & LIMITATIONS

While the formalization of our system guarantees sound properties (soundness of type systems, semantic simulation, and end-to-end safety), several design limitations must be made explicit.

Surface Language & Monomorphization Role. The source calculus $\lambda_{\mathcal{O}}$ is designed as a static construction calculus: it does not have an autonomous operational semantics for dynamically evaluating overloaded calls **ext** $x[\tau^k]$. Monomorphization is therefore not merely an optional optimization pass, but an essential construction step required to execute the program in the core calculus $\lambda_{\mathcal{M}}$. Furthermore, our monomorphization eliminates ad-hoc overloading but preserves parametric polymorphic declarations (**let**), which can then be handled via standard instantiation during low-level code generation.

Code Size & Performance Considerations. The global specialization of each instance eliminates the indirection at runtime inherent in passing dictionaries or virtual tables, but introduces a risk of binary bloat (*code bloat*) when multiple combinations of concrete types are instantiated. The integration of implementation-sharing techniques for types with the same machine-level memory representation is a practical optimization for the future.

Polymorphic Recursion & Specialization Termination. Our formalization proves the existence of an environment Ω under the explicit assumption that the catalog covers the necessary call sites ($\text{COVERS}_C(C, e)$). However, in the presence of polymorphic recursion (for example, a function $f : \alpha \rightarrow \dots f(\alpha \times \alpha)$), the chain of required instantiated types becomes infinite, preventing the monomorphization from terminating. The present system does not establish an automatic static termination criterion for these pathological cases; in practice, it is necessary to restrict expressiveness to well-founded recursive forms.

Expressiveness: Higher Order Kinds. Our formalism is limited to types and variables of kind $*$ (first-order types). Extending the formalism to higher-order type constructors (kind $* \rightarrow *$) is a natural extension that enriches the expressiveness of $\lambda_{\mathcal{O}}$.

9. RELATED WORK

The management and formalization of ad hoc polymorphism and overloading have a long history in the theory of programming languages [CW85, Rém02].

Type Classes & Dictionary-Passing. The canonical approach to statically typed functional programming is based on *type classes*, introduced by Wadler and Blott [WB89]. Their implementation using dictionary pass-through [Aug93] offers high modularity and allows for separate compilation, but introduces a potential overhead of indirection at runtime that subsequent specialization and inlining passes seek to eliminate when these optimizations are applicable. The qualified type system developed by Jones [Jon95] generalized this principle to higher-order polymorphism and type constructors. Our approach differs in that it avoids any explicit dictionary passing during evaluation: resolution is performed through global monomorphization, which is guaranteed by design.

Constraint Solving and Inference for Overloading. The integration of overloading into Hindley-Milner inference has given rise to multiple formalizations. Stuckey and Sulzmann [SS05] proposed a unified theory of overloading by coupling the $\text{HM}(X)$ framework with *Constraint Handling Rules* (CHRs). Recently, Beneš and Brachthäuser [BB25] revisited the very essence of overloading via flux-oriented variational typing, in order to strictly decouple constraint collection from instance selection. Other works have explored inference via semi-unification [FV05] or in the context of functional-logical languages [NTGS02]. Our

system relies on a catalog structure and a deterministic "most specific" extension criterion, ensuring the uniqueness of typing and the absence of ambiguity.

Overloading Calculus and Static vs. Dynamic Linking. Foundational calculi have sought to integrate overloading directly at the level of terms, as exemplified by Castagna's $\lambda\{\}$ calculus [Cas95], which unifies lambda abstractions and overloaded functions with late binding. Camarão and Figueiredo [CF98] developed type systems for overloading that depends on the context of use. In contrast to dynamic systems where resolution occurs at runtime, as in certain object-oriented languages [Sav10], languages such as C++ [KS13] also resolve overloading at compile time, but their resolution mechanism is not designed around the same formally verified metatheoretical framework as the one presented here.

Mechanized Verification. Finally, the formalization and automation of overloading typing in proof assistants have been the subject of recent work, such as that by Charguéraud et al. [CBR24]. Our work follows in this vein by providing a complete Coq/Rocq mechanization covering the correctness of the source system, the existence of monomorphization, forward simulation, and end-to-end safety between polymorphic and specialized programs.

10. CONCLUSION AND FUTURE WORK

In this article, we have presented a formal framework for static function overloading that reconciles surface expressiveness with the structural elimination of dictionary lookups during overload resolution. By formalizing the surface language $\lambda_{\mathcal{O}}$ and its translation via global monomorphization to the target calculus $\lambda_{\mathcal{M}}$, we show that global specialization of overloading can be formulated within a fully mechanized formal framework.

Our contribution is based on a complete formalization in the Coq/Rocq proof assistant (archive `meta-theory/LangD/`), free of any non-standard axioms. We have established the uniqueness of typing, the determinism of selecting the most specific extension, the preservation of typing under elaboration, forward semantic simulation (`simulation_multi`), and an end-to-end soundness theorem (`end_to_end_soundness`) guaranteeing that no monomorphized target program reaches a deadlock state.

Several promising avenues for future work are opening up:

- **Higher-order kinds and property hierarchies:** Extend properties to type constructors (`kind *  $\rightarrow$  *`) to model rich functional abstractions (functors, monads) and allow sub-property constraints.
- **Beyond the closed world:** Exploring hybrid compilation schemes combining intra-module monomorphic specialization and dictionary-based dispatch at module boundaries to enable separate compilation.
- **Mitigating the binary explosion:** Implementing memory representation sharing optimizations (similar to Rust or Swift) to limit the growth in compiled code size.

REFERENCES

- [Aug93] Lennart Augustsson. Implementing Haskell overloading. In *Proceedings of the Conference on Functional Programming Languages and Computer Architecture (FPCA '93)*, pages 65–73. ACM, 1993. doi:10.1145/165180.165191.

- [BB25] Jiří Beneš and Jonathan Immanuel Brachthäuser. The Simple Essence of Overloading: Making Ad-Hoc Polymorphism More Algebraic with Flow-Based Variational Type-Checking. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3286–3312, October 2025. doi:10.1145/3763168.
- [Cas95] Giuseppe Castagna. Unifying overloading and λ -abstraction: $\lambda\{\}$. *Information and Computation*, 117(1):1–37, 1995. doi:10.1006/inco.1995.1026.
- [CBR24] Arthur Charguéraud, Martin Bodin, and Louis Riboulet. Typechecking of Overloading in Programming Languages and Mechanized Mathematics. *Proceedings of the ACM on Programming Languages*, 8(POPL):1–31, 2024. doi:10.1145/3632865.
- [CF98] Carlos Camarão and Lucília Figueiredo. A Type System for Context-dependent Overloading. *Electronic Notes in Theoretical Computer Science*, 14:52–61, 1998. doi:10.1016/S1571-0661(05)80229-4.
- [CW85] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, December 1985. doi:10.1145/6041.6042.
- [FV05] Lucília Figueiredo and Cristiano Vasconcelos. Type Inference for Overloading by Semi-Unification. *Electronic Notes in Theoretical Computer Science*, 132(1):149–164, 2005. doi:10.1016/j.entcs.2005.01.025.
- [Jon95] Mark P. Jones. Functional programming with overloading and higher-order polymorphism. In Gerhard Goos, Juris Hartmanis, Jan Leeuwen, Johan Jeuring, and Erik Meijer, editors, *Advanced Functional Programming*, volume 925 of *Lecture Notes in Computer Science*, pages 97–136, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg. doi:10.1007/3-540-59451-5_4.
- [KS13] Brijender Kahanwal and T. P. Singh. Function Overloading Implementation in C++. *arXiv preprint arXiv:1309.5781*, September 2013.
- [NTGS02] Matthias Neubauer, Peter Thiemann, Martin Gasbichler, and Michael Sperber. Functional logic overloading. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '02, pages 233–244. ACM, 2002. doi:10.1145/503272.503294.
- [Rém02] Didier Rémy. Using, understanding, and unraveling the OCaml language: From practice to theory and vice versa. In *Applied Semantics: Advanced Lectures*, volume 2395 of *Lecture Notes in Computer Science*, pages 413–536. Springer Berlin Heidelberg, 2002. doi:10.1007/3-540-45699-6_11.
- [Sav10] Anthony Savidis. Integrated implementation of dynamic untyped object-based operator overloading. *Software: Practice and Experience*, 40(12):1079–1107, 2010. doi:10.1002/spe.1025.
- [SS05] Peter J. Stuckey and Martin Sulzmann. A theory of overloading. *ACM Transactions on Programming Languages and Systems*, 27(6):1216–1269, November 2005. doi:10.1145/1108970.1108974.
- [WB89] P. Wadler and S. Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '89, pages 60–76, New York, NY, USA, January 1989. ACM. doi:10.1145/75277.75283.